

Grounded Local RAG

One-page engineering explanation | FastAPI + local vector store | 18 September 2026

1. Design parameter: chunk size

I chose **180-word chunks with a 40-word overlap**. The task's reference files are likely short but factual: 180 words usually holds one procedure or policy together, while 40 words protects an answer that crosses a boundary. The chunker is an inspectable word-window function rather than a hidden framework abstraction; each exact chunk and ID is persisted in `data/chunks.json`.

2. Failure observed while building

During verification, two overlapping `pip install` processes tried to update the same Pydantic metadata file and raised `OSError: [Errno 13] Permission denied for pydantic-2.13.5.dist-info/INSTALLER`. The cause was concurrent environment setup, not the API; I let the first install finish and reran the second. In the product path, malformed-PDF parser failures are also caught and returned as a clear 422 response (covered by a test).

3. Metric tracked: local query latency

I ran `python scripts/benchmark.py` with 20 identical questions after ingesting the sample handbook. On this machine it recorded **0.81 ms median** and **2.40 ms p95** local query latency (excluding server/network overhead). That says the lightweight fallback is quick enough for a demo; it does **not** prove retrieval accuracy. Returned cosine scores and excerpts make each retrieval decision inspectable.

4. Not finished and next step

The offline fallback is hashed lexical embeddings, not a semantic local model. I did not build a labeled retrieval benchmark, OCR for scanned PDFs, metadata filtering, or multi-user concurrency. Next I would build a 30-50 question gold set, compare OpenAI embeddings against a local sentence-transformer using `recall@k` and faithfulness checks, tune the evidence threshold from those results, then add OCR only if target inputs require it.

Honest scope: runnable offline by default; OpenAI calls are optional and guarded with fallback/error handling.